Meerkat: Ranking Alert Families for Capacity-Limited Triage in Multi-Detector Security Operations

Jiacheng Wang

July 2026

Abstract

By the time an alert exists the detection has already happened; the question left to the analyst is which alerts to open first.

A Tier 1 analyst works a shift with capacity for perhaps tens of investigations, while one monitored company in the dataset used here emits several thousand alerts on a day containing no attack at all. A second imbalance sits inside the alerts themselves. A single step of an intrusion can produce thousands of near-identical alerts, so the loudest activity on the network is not the most important, and a system that ranks by apparent urgency will spend the analyst's entire day inside one event. Detectors disagree with each other: they use incompatible severity scales, describe the same machine under different names, and provide nothing that indicates whether two alerts belong to the same incident. Neither do they indicate when an alert is the only visible trace of a quiet step such as a privilege escalation, which produces a handful of records and matters more than the thousands beside it.

A professionally equipped SOC buys this layer. A commercial stack correlates alerts into incidents before an analyst sees them and ranks the result: Microsoft's GraphWeaver builds the incident graphs and Adaptive Incident Prioritization scores them, Sophos' TEQ predicts actionability from analyst escalations, and research systems such as AlertBERT, RAPID and DeepCASE group and rank with context. All of it assumes the grouping layer exists. A team without the budget for that stack runs open-source detectors instead, Suricata, Wazuh and a log anomaly detector here, and gets no such layer, so it must decide for itself what a single unit of work is. That situation, tackled with the resources it actually has, is the one this report is about.

This report describes Meerkat, a triage pipeline built on that stack, and treats the choice of unit as the primary design variable. Three successive designs are evaluated by leave-one-environment-out testing across the eight simulated companies of the AIT-ADS dataset, and each is measured by how much distinct attack activity is exposed within a fixed daily review budget, meaning the number of cases an analyst has time to open in a shift. The shipped model refits on a client's own incident records behind a gate that discards any refit that does not beat it on held-out incidents, and the confidence shown beside each queue item is the analyst's own escalation record; an earlier version showed a calibrated model output. The designs that failed are reported with the measurements that retired them, together with a metric that was withdrawn after it was found to measure redundancy while being read as quality.

1 Introduction

1.1 Motivation

This work began because I aspire to work in a Security Operations Centre. I have never observed one from the inside, and I had no evidence that I could do the work.

Reading about the role taught me its vocabulary; the difficulty only became visible once I built the system. Accounts of Tier 1 triage agree on what the analyst does and say little about which part is hard. I judged that the most reliable way to find out was to build the system myself: take the output of real detectors and produce the short list a human would work from.

The exercise became a research problem at the point where I measured it properly. My first design ranked individual alerts by predicted importance and performed well on the metrics I had been taught to report. It was useless in operation, and the reason had nothing to do with the model.

What the measurements showed is that the difficult part of triage lies before the model. It lies in deciding what counts as one thing to look at, and in deciding what counts as doing the job well. Under a fixed review capacity those two choices govern the outcome more than the choice of classifier does. That conclusion is what this report argues.

1.2 Four observations that shaped the design

Alerts are not independent. One step of an attack can produce thousands of alerts: a single directory-scanning tool in this dataset generated 7,068 against one host in about a minute. Treating those as 7,068 separate decisions consumes the whole review capacity and makes the loudest activity appear the most important.

Ranking alerts optimises the wrong objec-

tive. The dataset marks each attack step with an *attack window*, a time interval on a host, and the windows differ enormously in noise: four scanning windows hold 96.1% of all in-window alerts while a privilege escalation window may hold a handful of records. An objective defined per alert is therefore supplied almost entirely by the loudest few windows, and a quiet one can be missed at almost no cost to the training loss. Class balancing does not help, because the imbalance at issue is among the positives.

Importance is not well defined until the metric is. An analyst with time for ten cases does not want ten alerts describing the same burst, so per-alert precision rewards a queue that repeats itself. This report measures how many *distinct* labelled events a day's queue exposes. The choice is a proxy: it treats every attack step as equally worth reaching, and measuring what genuinely matters would need asset criticality and business impact, which this dataset does not carry and which I have no operational experience to supply. Section 9 states what it leaves out.

The unit of work is the primary design variable. What the system scores matters more than how it scores. Most of the improvement reported here follows from changing what a single row represents; the model above it changed little.

1.3 Related work

Published triage systems mostly assume the ordering problem is posed over units that an earlier stage has already assembled. Microsoft's Adaptive Incident Prioritization scores incidents that Defender has already correlated [4], and GraphWeaver is that correlation layer itself, joining related alerts into incident graphs at production scale [5]. Sophos' TEQ predicts actionability at alert and incident level using analyst escalations as labels [6]. AlertBERT learns to group alerts belonging to one attack [7]. RAPID ranks alerts using surrounding context [8], and DeepCASE analyses sequences of events for the same purpose [12]. Earlier work established analyst feedback and rarity as core signals [10], and a recent survey maps the space of prioritization criteria and methods [9].

An open-source deployment running Suricata, Wazuh and a log anomaly detector has no correlation layer at all. It has to choose its own unit of work, and this report treats that choice as the primary design variable.

1.4 Contributions

The report is organised around three claims, each supported by a measurement reported later:

- A unit of work built for capacity-limited review: a *session*, one detector rule firing on one host until that host falls quiet for a measured gap, grouped into a *family*, one rule on one host across one day. The grouping removes the volume imbalance structurally, and changing the unit is worth more than any model change measured here.
- A ranking stack matched to that unit: a per-session forest under a family re-ranker, beating every untrained ordering at every budget that separates rankings, at about a seventh of the alert volume of the ordering whose coverage comes closest. Its feature set, model class and shipped weights are each closed by measurement, and a client refits it on their own incident records behind a gate that discards any refit that does not win there.
- A measured negative result: under a fixed review budget, per-alert ranking cannot reach the quiet phases of an intrusion whatever the scorer, and per-item scores such as precision can rise while the queue gets worse.

2 Dataset

Experiments use AIT-ADS [1], published by the Austrian Institute of Technology under CC BY 4.0 and derived from the AIT log dataset [2]. It contains eight independently simulated enterprise environments, each monitored at the same time by Suricata (network intrusion detection), Wazuh (host intrusion detection) and AMiner (log anomaly detection). Each environment is subjected to a scripted multi-step attack made up of network scanning, service scanning, web enumeration (*dirb* and *wpscan*), web shell upload, credential cracking, reverse shell, privilege escalation, service disruption and DNS exfiltration.

The raw archive holds 2,655,821 alert records. Suricata alerts are collected by Wazuh and appear twice, once decoded from `eve.json` and once re-parsed from `fast.log`. Removing the lower-information duplicates leaves 2,349,098 alerts across 79 labelled attack windows. The 306,723 removed records were checked and contain no event-labelled positive.

Two properties motivate the choice of dataset. Three concurrent detectors make cross-detector normalization a real problem, and eight independent environments allow training on seven while testing on one the model has never seen.

2.1 Label semantics

Each alert carries two labels produced by different processes, and the difference between them decides what can honestly be claimed. The field `time_label` names the attack phase active when the alert fired and records timing alone: an antivirus update that runs during attacker scanning receives the scanning label while being entirely benign. The field `event_label` names the attack event the alert could

Detector	Alerts	Share	Names	Sev.
Wazuh	1,986,905	84.6%	24	3–10
Suricata	306,635	13.1%	29	1–3
AMiner	55,558	2.4%	34	none

Table 1: *The three detectors across all eight environments. The severity column gives the range actually observed.*

be traced to, and the official labelling rules cover only selected log sources, so an alert can sit inside an attack window, be caused by the attack, and carry no event label, while real exfiltration outside every recorded interval carries one.

Two aggregate properties govern everything that follows. First, event labels are heavily concentrated: `dirb` alone supplies 1,688,945 of 1,817,250 positives (92.9%). Second, and more importantly, the in-window alert population is concentrated the same way: four `dirb` windows hold 1,689,359 of the 1,758,559 in-window alerts, or 96.1%. Any objective defined over individual alerts therefore optimises mostly for one phase.

A window counts as **strictly reached** only when the queue holds an item containing an officially event-labelled alert from inside that window. Time overlap alone does not count. Of the 79 labelled windows only 60 contain such an alert, so 60 is a ceiling imposed by the supervision, and all coverage figures are read against it.

Finally, 72,212 event-labelled alerts fall outside every recorded window. The absence of a phase label therefore cannot be read as benign, and an absent event label means unlabelled.

2.2 What each detector produces

The differences between the three sources drive most of the normalization work and explain why a single severity ordering cannot be trusted.

Table 1 summarises the three sources. The volumes are strikingly uneven: Wazuh supplies 84.6% of all alerts and AMiner 2.4%. The alert vocabulary is small in every case, between 24 and 34 distinct alert names, and in this corpus each name corresponds to exactly one rule identifier, so naming and rule identity carry the same information.

The severity column is the first obstacle. Wazuh rule levels run from 0 to 15 with higher meaning more severe, and only levels 3 to 10 occur here. Suricata uses 1 to 3 with the ordering inverted, so 1 is the most urgent. AMiner emits no severity at all, because a log anomaly detector has no notion of rule gravity. Any cross-detector ordering therefore requires an explicit reconciliation, and the choice made for the detector that reports nothing is consequential: an early version assigned AMiner zero, which ranks an entire detector last by construction.

2.3 Which detector sees what

The three tools are complementary. No field group is populated by all three detectors, so any feature built on a specific field is a feature about one detector unless absence is modelled explicitly. The evidence an analyst wants is split across sources: Suricata alone reports flow volumes and TLS fingerprints, AMiner alone names the log file that triggered the anomaly, and only Wazuh supplies an ATT&CK tag, for a fifth of its alerts. The display side of the tool therefore cannot be organised by detector, because the same investigative question is answered by different sources depending on the alert.

2.4 Where the positives are

The concentration by phase was given above. It is matched by a concentration across detectors: Wazuh supplies 93.4% of all labelled alerts, Suricata 4.0% and AMiner 2.5%.

A model trained on individual alerts therefore inherits two biases: it learns predominantly about a single attack phase, and predominantly from a single detector. Both disappear when the unit of work changes, which is the subject of Section 3.2.

Timestamps are also coarser than the data volume suggests. Wazuh reports whole seconds almost always, and 72.2% of alerts show a zero interval to the previous alert on the same machine. No feature depending on fine-grained ordering within a second could be built, which rules out sequence models of the kind DeepCASE applies [12].

2.5 A richer dataset that was deliberately not used

Microsoft's GUIDE dataset is the obvious alternative and deserves an explicit account of why it was not used [11]. Drawn from real Defender telemetry, larger by orders of magnitude, and since July 2026 carrying expert prioritization labels with a queue-ranking protocol, it is by size, realism and label quality the better resource. Three things make it the wrong instrument here. Defender has already correlated the raw evidence into incidents, so the central experiment of this report, comparing units of work, cannot be run on it. It is one vendor's unified schema, so the cross-detector disagreement that Section 2.2 shows is the actual problem does not appear in it. And its ranking labels were published after this evaluation was fixed, so the comparison is left for future work. GUIDE is the natural next test: the unit construction and the coverage objective could be evaluated against its protocol directly, and the result would say whether the choice of unit still matters when the labels come from analysts who were there.

3 Method

3.1 Normalization and asset identity

The three detectors emit incompatible JSON, with the field coverage reported in Section 2.2. Wazuh nests rule metadata under `rule`, Suricata events arrive wrapped inside a Wazuh envelope at `data.alert`, and AMiner uses an `AnalysisComponent` structure with epoch timestamps. Normalization maps all three onto one common table, which is the field mapping a SIEM performs when onboarding a log source.

The first decision is what the shared table should contain: the intersection of what every detector supplies, or the union of what any detector supplies. Measured on this corpus the intersection is barely viable. Only twelve of the fifty-two columns are populated by all three detectors, seven of those twelve are identifiers this pipeline constructs itself, and what genuinely survives from all three tools is the timestamp, the alert name, the host string, the rule identifier and the requested web resource: a pipeline on those five fields could group alerts and little else. The union was taken, 43 of 52 columns carry values for at least one detector, and 26 columns come from exactly one tool, which is the argument for the choice.

Two conventions make the sparsity safe to model. A missing string is the empty string and a missing number stays missing, so that a field a detector never reports is not read as a measured value of nothing. Features built on these columns are then either shared by construction, such as the reconciled severity of Section 2.2, or carry an explicit presence indicator. This is also why the analyst-facing evidence panels are organised by what a field means: a panel is shown when it has something to show, which follows from the union.

Machine identity was the source of the most damaging defect in this work. Wazuh omits its `predecoder` block for agent-collected files, and an early implementation fell back to `agent.name`, the constant string `wazuh-client`, which collapsed 1,698,994 of 1,986,905 Wazuh alerts onto one fictitious host and put contextual counts off by three orders of magnitude. The pipeline now takes a customer-supplied asset inventory as an input and separates the machine an alert describes (`entity_id`, always an address the detector itself reported), the sensor that observed it (`observer_id`), and whether the machine belongs to the company. Suricata endpoints resolve against the inventory destination-first; AMiner alerts are attributed through the log forwarding path, which corrects the cases where AMiner names the collector. An earlier pass that inferred the inventory from the alert stream was removed, because nine addresses had their only hostname evidence inside an attack window.

The official configuration decides membership and asset role. The configuration disagrees with the identity Wazuh reports on 80,183 of 287,911 alerts (27.9%), in every case a permutation of external mail server names inside one environment. Group membership survives the permutation, so role-derived features stay valid, and display names come from the detector.

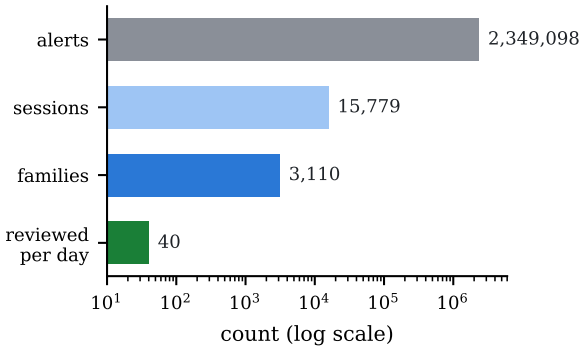

Figure 1: *What grouping removes. Two and a third million alerts across the eight environments become 15,779 sessions and 3,110 families, of which an analyst reviewing ten a day opens forty over the four days of one environment. The horizontal scale is logarithmic.*

3.2 From alerts to sessions to families

The unit scored by the ranker is the primary design variable of this work. The final pipeline groups in two stages.

A **session** is one detector rule firing on one host, closed once that host has been silent for longer than a gap parameter. A **family** collects a day's sessions sharing one host, one detector and one rule:

(day, `entity_id`, `detector_source`, `rule_id`).

The family is the unit an analyst adjudicates. The decision "this rule fired 380,023 times on this host today" is one judgement. The session exists underneath it because a family that spans a whole day mixes separate bursts of activity, and the analyst usually wants the burst.

Table 1 gives the reduction. The grouping also removes the volume imbalance structurally: each `dirb` burst becomes one row, so the 96.1% concentration reported above does not survive the change of unit. A family is positive when it holds at least one event-labelled alert, giving a positive rate near 11% that is stable across folds.

3.3 Where the family unit fails

Collapsing thousands of alerts into one decision has an obvious risk: if a family holds seven thousand routine alerts and one malicious one, the analyst sees a summary of a mostly boring group. It is rarer than it sounds because the rule is part of the family key, and different steps of an intrusion generally trip different rules, so each lands in its own family. Dilution

Scheme	K=5	K=10	K=25
Daily family only	27	46	53
Gap 10 s	24	27	31
Gap 60 s	37	44	56
Gap 600 s	**36**	**48**	**58**
Gap 3,600 s	39	44	56
Gap 21,600 s	38	47	58

Table 2: *Unit-boundary sweep. Windows reached of 79, single seed, 50 trees, under the sweep's own configuration, so rows compare to each other and to nothing else.*

requires the malicious activity to trip *the same* rule as the noise around it.

Measured on the demonstration environment, 47 families contain at least one alert the dataset traced to the attack, and in 42 of them the attack accounts for more than half the family. Dilution is the exception. Two families are heavily diluted: one carries a single traced alert among 254, the other three among 178, both under the same Suricata rule. Neither reached the queue.

Three properties push against the failure: sessions split a family wherever activity pauses, so malicious activity separated in time gets its own score; the best child score carries the re-ranker's heaviest weight, so one strong session raises its family; and several aggregates are maxima, so one severe alert propagates. All three are defeated by the same case: an alert interleaved in time with the noise, at the same severity, under the same rule, leaves no trace in any aggregate the model reads. The first of the two diluted families above is exactly that. This is a real cost of the family unit, paid for the reduction in Figure 1, and it is not mitigated anywhere in the current pipeline.

3.4 Choosing the session gap

The gap is a real parameter and was chosen by measurement. Six schemes were compared on identical data, features, folds and scorer, with only the boundary rule changing: the plain daily family, and gap-closed sessions at 10, 60, 600, 3,600 and 21,600 seconds. The gap was selected by nested validation inside the training environments only, so the held-out environment never influenced the choice.

Table 2 reports the sweep. Ten seconds is degenerate: 72.2% of alerts show a zero interval to the previous alert on the same machine, because this data has no sub-second resolution, so the median unit becomes one alert and 53.5% are singletons. Ten minutes was the modal inner choice, winning in five of eight folds, and was frozen as a single constant because per-fold selection proved unstable across seeds while the constant did not.

3.5 Features

Sessions are described by volume, duration, alert rate, reconciled severity, whether the detector supplied an ATT&CK technique, inventory membership, and context counted per machine per day. Families add aggregates of their child session scores together with span, alert count, cross-detector activity on the same host, and asset role.

Raw identities never become features. No IP address, hostname, rule identifier or alert name enters the matrix as a value, so a model trained on seven companies cannot recognise the eighth by name. The feature vocabulary is fitted on training environments only, so a rule first seen in the held-out company is treated as unseen.

Two decisions correct defects of earlier designs. Rarity is a continuous count: the first design bucketed any alert name occurring fewer than fifty times into one shared category, and those 38 names, carrying only 744 alerts, touch 33 of the 79 windows including all eight privilege escalation windows. The phases the first design never reached are largely the phases whose distinguishing names were removed before training.

Severity reconciliation maps three incompatible native scales onto $[0, 1]$. Wazuh reports 0–15 where higher is more severe, Suricata reports 1–3 where lower is more severe, and AMiner reports nothing and takes the midpoint. Assigning AMiner zero, as an earlier version did, ranks a whole detector last.

3.6 Scoring the session

Three models sit on this representation, one per level of the structure. A Random Forest scores each session, a logistic re-ranker scores each family, and a calibrator converts the family score into a figure an analyst can read as a probability.

The session model is a Random Forest with balanced class weights, since roughly one session in nine contains a labelled alert. It reads only the transferable description of Section 3.5: volume, duration, rate, reconciled severity, rarity, inventory membership and per-machine context.

3.7 The family re-ranker

A family usually holds several sessions, so its score has to be assembled from theirs. The obvious way is to take the highest child score and discard everything else. That choice throws away every property of the group: how many bursts there were, how long the activity ran in total, how many alerts it produced, and whether any other detector was active on the same machine at the same time.

The re-ranker exists to use those properties. It reads thirteen aggregates together with indicators for the asset's inventory roles:

- the maximum, mean and spread of its child session scores, which describe both the strongest evidence and how consistent the sessions were;
- the number of child sessions, the total span in seconds and the total alert count, which separate a single tight burst from low-rate activity spread across the day;
- context counted per machine per day, and the number of distinct detectors active on that machine within ten minutes of the family, which is the corroboration signal. It is counted per machine, since the family key already fixes the detector;
- the spread of observable taxonomy attached to the family, counted over Suricata alert categories, ATT&CK techniques supplied by the detector and rule groups.

The training boundary matters more than the model choice. The child scores that the re-ranker learns from must be produced by a forest that never saw those sessions, because a forest scoring its own training data returns an unrealistically clean signal, and a re-ranker fitted on it would learn a relationship that does not hold at inference. Child scores are therefore generated by an additional leave-one-environment-out pass inside the training environments, and the re-ranker is fitted on those out-of-fold scores.

The re-ranker is a balanced logistic regression. The family table holds roughly three thousand rows across eight environments, which is little data for a second ensemble, and a linear model over standardised inputs keeps each coefficient inspectable, so the contribution of any one aggregate can be read directly. Balancing is applied because about one family in nine is positive.

The coefficients ship fixed, and that deserves its own justification because it sits in tension with the refit doctrine of Section 6. The weights encode how kinds of evidence combine, corroboration against volume against role, and those relationships held on every environment measured, including a dataset from a different testbed. What varies between environments is the scale of the incoming scores, and exactly that part, the standardising scaler, is refitted on the client's families at every retrain. A client with enough incident history can contest the coefficients themselves: Section 6 reports the measured threshold at which a locally fitted re-ranker becomes competitive, and the tool adopts a local fit only when it beats the shipped weights on that client's held-out incidents.

What the re-ranker is worth against the simpler alternative is reported with the results.

3.8 Confidence display

The re-ranker orders well. Its raw scores overstate certainty. On the held-out environment used for the worked examples it assigns scores above 0.999 to 25 of 326 families, and families scoring that high are genuine about 87% of the time. Presenting 0.999 to an analyst would invite a certainty the evidence does not support.

The first remedy was a one-variable logistic calibrator, fitted on out-of-fold re-ranker scores from the training environments only, mapping the score onto the frequency actually observed at that score. It worked, lowering the pooled Brier score from 0.0301 to 0.0202 and improving all 24 held-out environment and seed combinations. It was later removed from the tool, because the frequency it reports is a property of the training environments: a client reading 87% has no way to ask "87% of what", and the honest answer, of families in eight simulated testbeds, is not their environment.

What ships instead needs no model. The queue's `esc%` column reports, for families at the same score rounded to one decimal, how often the analyst's own past reviews escalated, with the count of reviews it rests on. A band shows nothing below five reviews, and a fresh environment shows the score alone. The number can be verified by hand from the tool's own decision export, and it grows out of the analyst's own history. Neither display was ever allowed to reorder the queue, which is ordered by the raw score alone, so a change to how confidence is shown can never change whose case is opened first. Two caveats are known and unmeasured: the display can feed itself, since an analyst who escalates more at a score raises that band's rate, and a retrain shifts the score distribution, so bands built from reviews of an older model describe scores the current one no longer emits.

3.9 Enrichment and queue

Families are mapped to ATT&CK techniques and tactics [3] through a three-tier mapping: a reviewed override where one exists, otherwise the detector's own tag, otherwise nothing. The mapping is rule-based so that every claim stays auditable. Only the vendor-supplied identifiers are used as a model feature. The hand-reviewed mapping was authored with knowledge of this dataset and is reserved for display, a decision backed by measurement: feeding it to the model lifted technique coverage from 403,026 to 1,747,323 alerts, the high-volume reconnaissance families then carried techniques too, and coverage fell while precision and NDCG rose. Two further widenings of the feature set failed the same pre-registered gate the same way.

The queue is the top K families per day, where K is a capacity setting a team chooses from the time it has.

4 Evaluation protocol

Evaluation is leave-one-environment-out: seven environments train, the eighth is held out, repeated eight times. The feature vocabulary, the forest, the re-ranker and the calibrator are all fitted without the held-out environment.

The primary metric is strict attack-window coverage at a fixed daily budget. It treats the queue as a set, so ten near-identical families do not score as well as ten families describing different activity. This is the practical Tier 1 question: with limited time, how many distinct suspicious activities did the queue expose?

What one unit of the budget costs. Because the designs compared here score different objects, K does not denote the same work in each. Under the first design it is K alerts; under the later two it is K families, and one family may contain thousands of alerts. The comparison is only meaningful if a family is genuinely a single decision.

The argument that it is rests on what the analyst is asked. A family poses one question, whether this rule firing on this machine today warrants investigation, answered from a summary; the accompanying tool opens a family that way, with the alerts available on request. It remains an unmeasured assumption about analyst effort. The forty families a ten-a-day budget selects on the demonstration environment hold a median of 41 alerts, a ninetieth percentile of 686 and a maximum of 7,068, so two in five are plainly one decision and the largest is arguably one decision that is not obviously the same amount of work. No practitioner was timed, so this report measures what a queue exposes. The cost of working through it is unmeasured, and the gain attributed to changing the unit is overstated to whatever extent a large family costs more than a small one. What can be said without assumption is the compression: those forty decisions span 15,653 of the environment's 36,358 alerts, so 43 per cent of everything the detectors produced is represented in the queue.

The comparisons within the second and third designs are unaffected either way, since those score identical objects and differ only in how they are ordered.

Three protocol rules were adopted after an earlier iteration produced conclusions that did not survive scrutiny. Results are read on budget curves, because an earlier gate fixed at $K{=}50$ had no derivation from analyst staffing. Conclusions are restricted to $K \in \{5, 10, 25\}$, reviewing 6.4%, 12.9% and 32.2% of the family table, because larger budgets review most of what exists and cannot separate rankings. And every design is compared against orderings a team could write without a model, native severity, rule rarity, family size and random order, the last because a metric that a shuffled queue scores well on is not measuring much.

Dominance first, sign tests where they apply. With eight folds sharing seven training environments each, paired tests are neither independent nor adequately powered, so a method is accepted only if it is at least equal at every unsaturated budget and strictly better at one. Where two orderings are scored on identical folds, an exact sign test over the fold-level differences is reported beside the dominance claim, and a comparison with fewer than six discordant folds is reported as undecidable.

Coverage is reported beside its cost. Coverage alone rewards big queues: an ordering that fills the budget with the largest families reaches many windows only because it holds most of the day's alerts. Every headline row therefore carries the number of alerts inside the queue and their share of the day, and a floor row reviews the whole day as a single item, which is the cheapest way to reach every window and the upper bound the cost column exists to expose.

The forest size was swept at 50, 200, 300 and 500 trees and the re-ranker at three seeds. Two hundred trees is reported throughout, and Section 9 records how far the results move across that sweep.

Attack windows are known only during offline evaluation, and that separation is checked. Triaging one environment with the label files supplied, and again with them withheld, produces identical scores and an identical queue. The largest difference in any score is 2.2×10^{-16}, which is floating point noise. The only columns that change are evaluation columns no model reads.

5 Results

5.1 First design: ranking individual alerts

The first design encoded each alert as 35 numeric columns dominated by activity counts over 60, 600 and 3,600 second windows, and trained two Random Forests: a *context model* on attack-window membership and an *event model* on per-alert event labels, fused by maximum.

On conventional metrics it looks strong: 86.44% alert-level recall at 54.13% workload reduction and 98.87% average precision. At a fixed review budget it fails, and Table 3 shows how, at 50 reviews per day.

Broken down by phase, the failure has a shape. The design reaches the two web enumeration phases, which are the loud ones, and no variant reaches a single web shell, privilege escalation, credential cracking, reverse shell or network scanning window at all, at a budget five times larger than the final design uses. During those steps the intruder is already inside, so those are the windows a triage system needs to surface.

Variant	Windows reached
Event model	26
Native detector severity	21
Full hybrid pipeline	20
Event and context fused	20
Context model	14

Table 3: *First design. Windows reached at 50 reviews per day, summed over eight held-out environments, of 79.*

The second row of Table 3 is the result that redirected the project. Sorting alerts by the severity the detectors already assign, a rule requiring no model, no training data and no features, reaches more attack windows than the complete learned pipeline built to replace it. Name rarity, an equally cheap heuristic, behaves comparably. A learned score that cannot beat the number already printed on the alert adds nothing, and no amount of tuning at the alert level changed that ordering.

Fusing the two scores by maximum costs six windows against the better component, because the dense context signal pushes event-ranked alerts out of a capacity-limited queue.

Two measurements identify the cause. First, 96.1% of in-window alerts belong to four `dirb` windows, so an objective over alerts is overwhelmingly an objective about one phase, and the 86.44% recall above is largely a measurement of that phase. Second, about 93% of the context model's importance falls on activity-volume counters, which is the representation the data invites.

The conclusion is that the binding constraint is the scored unit.

5.2 Second design: alert families

Changing the unit invites an obvious question: does the queue still need a learned model, or would something simpler do? Seven orderings of the same families were therefore run on identical data, folds and budgets, so that any difference between them comes from the score alone.

They are four heuristics that need no training (family size, name rarity, reconciled native severity, and a learned per-rule prior), an unsupervised Isolation Forest anomaly score, the same score handed to the supervised model as an extra feature, and a Random Forest. The forest was chosen for the shape of this problem: roughly three thousand families with around eleven per cent positive is little data, forests need no tuning at that size, they handle features on wildly different scales, and they can be inspected feature by feature, which matters when the output justifies itself to an analyst.

Table 4 reports the result.

Budgets of fifty and one hundred were also measured and are omitted here. Past the saturation threshold of Section 4 every ordering reaches 59 or 60 windows, random order included, so the columns carry no information.

Ordering	K=5	K=10	K=25
Random Forest	**34**	**43**	55
Random Forest + anomaly	30	41	**56**
Isolation Forest anomaly	24	30	44
Native severity	19	26	48
Family size	23	30	39
Name rarity	18	27	44
Per-rule prior	18	27	42
Share of table reviewed	6.4%	12.9%	32.2%

Table 4: *Second design. Attack windows reached, of the 79 the dataset labels, when the same alert families are put in different orders. K is the number of families reviewed per day.* Anomaly *is an Isolation Forest score, which measures how unlike the others a family looks across all its features without using any label; it appears once as an ordering of its own and once as an extra input to the supervised model. The final row gives the share of all existing families that each budget reviews, which is why larger budgets stop discriminating.*

One choice behind these numbers: a family counts as positive during training only when it holds an alert traced to the attack. Counting mere window overlap would add several hundred positives and was measured slightly worse, because time overlap is not evidence: the antivirus update of Section 2.2 runs while the attacker scans, and the looser label teaches the model that routine activity at an unlucky moment is worth surfacing.

The supervised ranker meets all three criteria fixed in advance. It beats family size, name rarity and the per-rule prior at every budget that can separate rankings, and it beats severity and the Isolation Forest as well. Here the learned score beats every heuristic; at alert level it lost to severity.

Two smaller findings sit underneath that. The Isolation Forest reaches more windows than every hand-built heuristic, so looking at a family across all its features finds more than looking at any single one, even with no labels. Handing its score to the supervised model as an extra feature makes it slightly worse: the supervised model had already extracted whatever the unsupervised one found.

Gradient boosting was considered at this point and deliberately not run: the categorical fields it is best at are exactly the identities excluded from the feature matrix, and on the numeric features alone a tie with the forest was predicted. The question was later settled by measurement, and Section 5.7 reports the result: the prediction was wrong, in the forest's favour.

Ranking method	K=5	K=10	K=25
Learned re-ranker	**52**	**58**	**58**
Best single child session	44	53	58
Family size (alert count)	29	30	39
Rule rarity, rarest first	23	32	47
Native detector severity	19	33	46
Random order	17	29	43

Table 5: *Third design. Windows strictly reached of the 60 that the labels allow, summed over the eight held-out environments. The two model rows are three-seed means at 200 trees; the four below them need no training and are scored directly, with random order averaged over three seeds.*

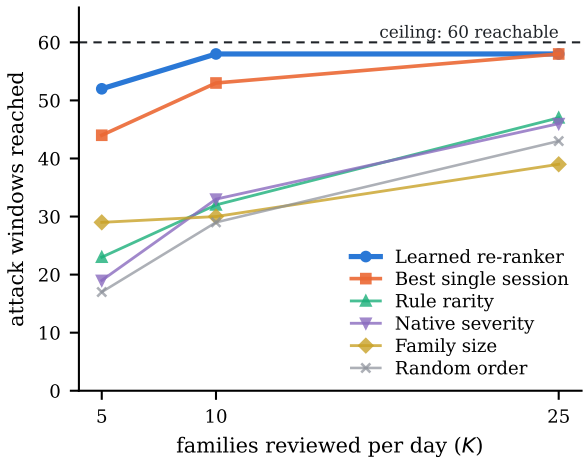

Figure 2: *Attack windows reached as the daily review budget grows. The dashed line is the ceiling of 60 windows the labels permit. The learned re-ranker saturates at ten families a day, two windows short of the ceiling, while the untrained orderings are still climbing at twenty-five.*

5.3 Third design: sessions and a re-ranker

The third design keeps the family as the unit an analyst reviews and changes how that family is described and scored. It splits each family into the sessions of Section 3.2, so a rule firing in three separate bursts keeps that shape, and it replaces the rule for combining those sessions with the learned re-ranker of Section 3.7. Table 5 compares it against every alternative ordering of exactly the same families, under the same budget and the same counting rule, so the only thing that varies is the score.

The rows below the model are the same alternatives as before, now applied to the two-level representation. *Best single child session* is the simplest way to score a family without a second model, taking its highest-scoring session and ignoring the rest. The three after it order by raw volume, by how unusual the rule is, and by the priority the detectors themselves assign. Random order is the floor.

Figure 2 shows the same numbers as a budget curve. Two comparisons carry the result. The re-ranker against best single session isolates what reading the whole family is worth, since both see identical families and differ only in how the child sessions are combined: eight windows at a budget of five. The re-ranker against native severity, 52 against 19, is what the pipeline adds over simply trusting the number the detectors already print on the alert, and it is the comparison that failed at alert level.

Ranking method	Steps	Items/day	Alerts/day
Learned re-ranker	**58**	**10**	**5,775**
Best single child session	54	10	42,830
Family size (alert count)	30	10	55,531
Native detector severity	33	10	4,542
Random order	32	10	9,125
Whole day as one item	60	1	57,295

Table 6: *Per evaluated day, 41 days over the eight held-out networks. Steps are summed over the networks, of the 60 findable; the alert column is what sits underneath the day's queued items; the final row reviews each day as a single item.*

Coverage also saturates at ten: reviewing twenty-five families a day reaches no more windows than reviewing ten. The saturation is a property of this testbed's sixty findable windows, and it carries no operational advice for a real network.

Table 8 restates the comparison as a working day. One queue item is a family, every alert of one rule on one machine over one day, so the key keeps the item uniform and one judgement usually settles it; the sessions inside are opened when it does not. The day's workload is ten of these summaries, and the alert column is what sits underneath them, opened on demand while investigating. An average evaluated day holds 57,295 alerts.

The sign tests separate the re-ranker from severity, family size, rarity and random order on every seed (p from 0.008 to 0.031); against the best single child session they are undecidable on coverage, with identical windows at twenty-five and three of eight folds differing at ten, and the alert column separates them: about 4,100 alerts underneath the queue per step reached, against 32,500 for the nearest ordering with comparable coverage. Severity is the one cheaper row and reaches 33 of the 60 steps.

The budget counts decisions, and every family method makes ten a day; the alert column counts the material underneath those decisions. Against the per-alert first design only the decision count is compared, and against best child, where coverage ties, the alert column is the tiebreak.

5.4 The queue on a quiet day

Most real days contain no attack, and the queue fills to K regardless: no abstention exists. What it fills with was measured on the demonstration environment, whose first three days are attack-free and whose last day carries the scripted attack. Table 7 reports

Day	Top	Median	Novelty items
First, no attack	1.00	0.27	5 of 10
Second, no attack	0.32	0.15	2 of 10
Third, no attack	0.35	0.09	2 of 10
Attack day	1.00	1.00	7 of 10

Table 7: *The daily top ten across the demonstration environment's four days. Novelty-style items are AMiner findings of the "new value seen" kind.*

Ranking method	Windows	Alerts	Day share
Learned re-ranker	**58**	**29,597**	**25%**
Best single child session	54	219,506	30%
Family size (alert count)	30	284,595	93%
Native detector severity	33	23,279	7%
Random order	32	46,767	18%
Reviewing everything	60	293,637	100%

Table 8: *What the queue costs to read at ten families a day. Windows are summed over the eight held-out environments; the alert and share columns are per-environment means, and the share is the mean of per-day shares, so neither column is a plain division of another. Aggregation follows the benchmark's summary tables, so coverage entries can differ by a window or two from the three-seed means of Table 5. The final row reviews each day as a single item and is the bound both columns are read against.*

each day's top ten.

The score column separates the days. After the first day, no attack-free queue item scores above 0.35 and the medians sit below 0.15, while the attack day queues ten items at 1.00, so an analyst reading the score column sees the difference between a quiet day and a loud one even though both queues hold ten rows. The first day behaves worse: AMiner's baselines are still new, half the queue is "new value seen" findings, and one scores 1.00. That pattern is a warning for production, where new-value rules fire on every patch cycle and configuration change; on a long-running network the settling period would recur at every change. An explicit quiet-day signal, a score floor under which the tool says the day holds nothing worth opening, does not exist and is future work.

Side by side at ten reviews a day and the identical metric, the best per-alert variant reaches 21 windows, alert families 43, and sessions with the re-ranker 58. Every design reviews the same number of items, so the improvement comes entirely from what those items are.

5.5 Coverage by attack phase

Table 9 breaks the final result down by scripted step.

In Table 9, the column headed *1st* carries the first design's result for comparison: no per-alert variant reached a single web shell, privilege escalation, credential cracking or reverse shell window, and that was at five times the review capacity. Two windows stay out of reach at any budget: one service stop and one network scan. Both are single-window steps on hosts

Phase	Reach.	1st	K=5	K=10	K=25
`wpscan`	8	3	7	8	8
`dirb`	8	5	7	8	8
`webshell`	8	0	8	8	8
`priv_esc`	8	0	7	8	8
`service_stop`	8	4	7	7	7
`service_scans`	7	1	5	7	7
`cracking`	7	0	7	7	7
`reverse_shell`	4	0	4	4	4
`network_scans`	2	0	1	1	1
Total	60	—	53	58	58

Table 9: *Windows reached per attack phase by the final design.* Reach. *is how many windows of that phase contain a labelled alert and can therefore be reached at all.* 1st *is the best any per-alert variant of the first design reached for that phase, at a budget five times larger; it is a per-phase maximum and so is not totalled; the remaining columns are the final design at each daily budget. Figures are the union over three seeds, so the K=5 column sums to 53; the three-seed mean is 52.*

Session forest, grouped importance	
Asset role from the inventory	34.3%
Activity on that machine that day	26.7%
Rule rarity	18.1%
Family volume and rate	11.1%
Reconciled severity	5.8%
Detector identity	3.9%
Native ATT&CK presence	0.1%
Family re-ranker, standardised weight	
Best child session score	+1.86
Mean child session score	+0.89
Rule group count	−0.77
Asset role: `dnat`	+0.68
Asset role: `servers`	+0.62
Alerts on the machine that day	+0.59
Detectors nearby, ten minutes	+0.52
Asset role: `firewall`	+0.50
Families on the machine that day	−0.50

Table 10: *What each layer relies on, measured over the eight held-out folds. Forest importances are grouped and sum to one hundred per cent. Re-ranker weights are standardised, so a positive value raises a family's position and a negative one lowers it. The lower block lists every input with an absolute weight of* 0.50 *or more; the rest fall below and are omitted.*

the queue is already surfacing for other reasons, so the analyst is looking at the machine even though that particular window is not credited.

5.6 What the model leans on

Both layers can be inspected, and Table 10 reports them. The forest figures are impurity importances grouped by what the features describe, averaged over the eight folds. The re-ranker figures are standardised logistic weights, so they are comparable with each other and carry a direction.

Configuration	K=5	K=10	K=25
With the inventory	51	58	58
No asset-role feature	46	53	58
Best ordering needing no model	23	33	48

Table 11: *Removing the inventory. Windows reached with every asset-role feature stripped from both layers, against the same configuration as shipped, at 50 trees and one seed. The final row is the strongest ordering that needs no model at all, included for scale; its figures come from this sweep and sit within a window of Table 5's untrained rows.*

Three things are worth drawing out. Asset role is the single largest group in the session model, and it recurs in the re-ranker, where five separate roles carry weights between +0.38 and +0.68 and one of them ranks fourth of all inputs. Both layers therefore lean on information the inventory supplies and no detector reports, which is what makes the ablation below worth running. Native ATT&CK presence contributes essentially nothing at 0.1%, confirming that its value in this pipeline is analyst-facing context; it contributes 0.1% to ranking. And the re-ranker still leans hardest on the best child session, which is the quantity the simpler baseline uses, with corroboration and asset role supplying the margin between them.

Asset role carrying the largest share raises a fair question. If the strongest signal in the model comes from a file the customer supplies, does the whole pipeline depend on being handed a good inventory? Removing every role feature from both layers and running the evaluation again answers it directly, and Table 11 reports the result.

The roles are worth five windows at the two budgets where capacity binds, and nothing at the largest: a real contribution, since with no inventory at all the pipeline still reaches 46, 53 and 58, ahead of every ordering that needs no model. Organisations that run a SOC usually keep an inventory, often stale and incomplete, and Meerkat asks less of it than commercial products do, using it only to identify machines and their roles. The caution of Section 9 applies: the eight environments share a topology, so a role may partly stand for "the kind of machine this script attacks", and the gap between 34.3% of decision weight and five windows of sixty is itself informative, because importance reports which features the model consulted; the ablation reports the coverage lost without them.

Two weights are negative in a way worth noting. A family carrying many rule groups, and a machine carrying many separate families that day, both lower a family's position. The plausible reading is that both mark ordinary busy machines, though impurity importances and linear weights are descriptive, and establishing that would need an ablation this report does not run.

The feature set itself was closed by a pre-registered ablation. Each of 34 removal masks, single features and correlated groups across both layers, was applied and the pipeline re-evaluated, under a gate written down first: a removal is accepted only if coverage does not drop at five or ten or in the budget area on any seed, no fold worsens beyond noise, and nothing drops on a second dataset. All 34 masks were rejected. The largest single dependency is rule rarity, whose removal costs 13, 8 and 9 windows at a budget of five across the three seeds. A lenient variant of the gate admits an 18-column model that matches the baseline on every AIT seed, and it was not adopted: three of the columns it removes carry signal on the AInception data of Section 7, and a feature that carries no signal on one testbed and measurable signal on another is kept for the environments not yet seen. The 28 columns ship as measured.

Arm	Seed 51	Seed 52	Seed 53
Random Forest	**53**	**50**	**50**
Forest + anomaly	54	52	52
Gradient boosting (HGB)	46	45	42
Isolation Forest alone	38	38	38

Table 12: *Model class at K=5, windows reached per seed under the final protocol. The anomaly feature's gain rests on two, three and one effective fold differences, below the six the sign test needs.*

5.7 Model class, measured

The tie predicted in the second design was later put to the test, under the final protocol, at 200 trees, on three seeds, with every arm reading the same features. Histogram gradient boosting, the Isolation Forest score as an extra feature, and the Isolation Forest alone were each run against the shipped forest, and Table 12 reports windows reached at a budget of five per seed.

Gradient boosting loses five to eight windows on every seed, so the predicted tie did not happen and the forest stays on merit. The baseline's spread across seeds, 50 to 53, sits inside the seed noise Section 9 describes at the tightest budget. The anomaly feature gains one or two windows per seed, and the gain rests on so few discordant folds that the sign test cannot call it real; it is recorded as future work. The Isolation Forest alone confirms the second design's finding at the final protocol: unsupervised structure reaches far less than supervision does.

6 Retraining on a client's records

The shipped model was trained where every alert carries a label. A deployment has nothing of the kind. What a SOC can produce is an incident record: an attack ran on this machine between these times, with a closing verdict. The retraining path is built for ex-

actly that input, a CSV of `start,end,host,verdict` rows exported from a ticketing system, with verdict names following OCSF's Incident Finding.

Weak labels. Nothing inside an incident window is asserted positive, because which burst was the attack is unknown. Every session overlapping an incident joins a bag, and the bag shares the weight of one label: with n sessions in the bag each carries k/n, and k=1 keeps every ticket's total influence at exactly one whatever its width. The distinction was measured: bag-level labels are 93.9% pure against 75.5% at session level, and k from one to five differs by less than seed noise over ten paired seeds ($p = 0.34$), so k=1 is chosen for having a reason: without the discount, one vague all-day ticket outweighs twenty precise ones, and in the worst environment a single window carried 48.5% of the positive signal. Sessions outside every incident are the negatives, unless a reviewed-periods file says they were never looked at, in which case they are unlabelled and train nothing. The default therefore reads unticketed time as clean, and a SOC's tickets record what it caught, so without the reviewed-periods file the model trains on the SOC's blind spots; providing it is the safer practice.

The gate. A refit is accepted by contest only. Five forests are fitted at different seeds on the earlier days; each scores the most recent `holdout` days, and a candidate passes only if it reaches at least as many held-out incidents as the shipped bundle and disagrees with it on at least six, because a two-sided sign test on fewer than six same-direction pairs cannot call a difference real ($2 \cdot 0.5^6 = 0.031$). A majority of the five must pass, the median forest is saved, because saving the best would select on the incidents the gate just spent, and a shipped-bundle-with-rescaled-re-ranker arm is reported beside the verdict so the client can see how much of any gain needed no labels at all. A refused retrain exits with its own code, reports every failed precondition at once, and saves nothing.

Measured end to end on the benchmark, ticket-style labels cost little: retraining from incident windows reaches 49 held-out incidents at a budget of five against 51 with per-alert ground truth.

Contesting the ranking weights. The re-ranker coefficients of Section 3.7 ship fixed, and a client can contest them: a flag fits the ranking weights on their own incidents, honestly through day-blocked folds, and the same gate adopts the local fit only when it beats the shipped weights on the held-out incidents. Measured on pooled benchmark history, a local fit reaches nothing on a single campaign and becomes competitive around fifteen positive families, so the tool states that threshold, proceeds, and lets the gate decide. The archive is kept whole between retrains: refitting on only the newest half of the days roughly halves coverage, so labelled history is trimmed only when the drift report of Section 7 says the old data no longer resembles the present.

7 Transfer and portability

What travels between environments was decided by measurement, and it settled into three layers. The units, the metrics and the gate are reused everywhere unchanged. All 28 feature definitions are computable from any of the three detectors' output. The fitted weights are refitted per environment, because a forest carried across environments loses to a local refit.

Two cross-dataset checks probe the boundary of that doctrine. On CAM-LDS, a dataset from a different testbed [13], the shipped re-ranker coefficients still beat the best-child ordering at a budget of five on every seed, 56, 56 and 55 windows against 47, 51 and 49 on the temporal metric, so the corroboration structure the coefficients encode is not an artefact of one testbed. On AInception [14], 378 of 1,351 network alerts carry ATT&CK identifiers embedded in rule metadata, which the pipeline reads natively, and three of the columns the lenient ablation would have removed carry measurable signal there, which is why the strict feature set ships.

The severity scale crosses detectors by fiat and was validated after the fact. Wazuh's 0–15, Suricata's inverted 1–3 and AMiner's silence map onto one $[0, 1]$ scale; within each detector the mapped value orders genuine against benign families with AUC 0.995 for Wazuh and 0.957 for Suricata, and the shared scale beats both a per-detector variant and a no-severity variant in the end-to-end evaluation. The choice made for the silent detector matters: AMiner takes the midpoint, because an early version that assigned zero ranked a whole detector last by construction.

Between retrains, drift is monitored. The population stability index is computed per feature against the training profile, with 0.10 read as stable and 0.25 as a major shift, alongside the share of rules the model never saw. A major shift exits with its own code so a script can act on it. Below roughly 300 training sessions the report is mostly noise, and the tool says so.

8 Discussion

One repetition mode survives the family unit and is left as a known cost. Much of what the model reads describes how busy a machine was, so a machine under heavy attack can take every place in the queue: seven of the eight environments contain a day where all ten selections come from one machine. A greedy selection trading score against the novelty of a family's machine, detector, rule and roles was measured, recovers part of the loss in the weakest environment at no cost in precision, reduces exactly to the ranked queue when the trade-off is switched off, and is not

shipped: the flat queue stays, and the repetition is a known cost.

The main result is that the choice of metric decided every major design outcome. Under precision or NDCG, the three widened feature sets look like improvements and the first per-alert design looks respectable at 86% recall. Under strict window coverage at a fixed budget, all four are correctly identified as worse. A team that measures triage by precision will narrow its queue and miss the quiet phases of an intrusion.

The second concerns infrastructure whose value depends on the formulation it serves. Machine identity and the inventory were built to repair the attribution defect of Section 3.1, after the per-alert design was abandoned, so their contribution under that design was never measured. Under family ranking the same work became substantial, 34.3% of the session model's weight and five windows in the ablation, and it paid off through asset context; the cross-detector corroboration that motivated it carries far less weight.

Relative to prior work, this pipeline occupies the position that arises when the correlation layer assumed by AIP [4] and by incident-level TEQ [6] is absent. The family key is a fixed, explainable substitute for the learned grouping of AlertBERT [7], chosen because an analyst can audit it and it needs no training. The grouping technique is standard. The contribution is the measurement: on this data the grouping is worth more than the ranker, and a widely used per-item precision metric can actively mislead.

A metric used earlier in this work, reported as "queue precision", turned out to correlate -0.991 with the number of attack windows reached: it measured redundancy while being read as quality. It has been withdrawn from all conclusions.

9 Limitations

The results above are bounded by four separate things: what the labels can express, what the dataset represents, what the model was never given, and what I was able to bring to the problem. They are set out in that order because the first constrains everything after it.

9.1 The supervision is weak

Every number in this report is measured against labels that are incomplete by construction, and this is the most serious limitation.

The dataset supplies two labels per alert. One names the attack phase active when the alert fired, which records timing alone: an antivirus update that happens during attacker scanning receives the scanning label while being entirely benign. The other names the attack event the alert could be traced to, and the official labelling rules only cover selected log sources, so an alert can sit inside an attack window, be caused by the attack, and still carry no event label. Section 2.2 gives a case of exactly that.

Three consequences follow. Nineteen of the 79 windows contain no event-labelled alert at all, so no queue can reach them under this definition and the effective ceiling is 60 of the 79. An absent label means unlabelled, so review rates outside windows are never reported here as false-positive rates, and any figure that reads like precision is precision against incomplete positives. And because the model is trained on these labels, it learns what the labelling process captured.

9.2 The data is simulated and unrepresentative in known ways

The eight environments come from one testbed running a shared attack script. They are consistent with each other in ways real companies would not be, so training on seven and testing on the eighth is a weaker test than it appears: the held-out environment is unseen yet familiar in shape. No claim about real-world performance follows from these results.

Attack prevalence ranges from 10.2% to 91.2% across environments, far above anything operational, where the great majority of days contain no intrusion at all. Timestamps are coarse, with 72.2% of alerts showing a zero interval to the previous alert on the same machine, which rules out any feature that depends on fine-grained ordering and closes off the sequence-based approach DeepCASE takes [12]. The attacks are scripted and run during simulated working hours, so a feature keyed on time of day learns the testbed's timetable; two such features were dropped before fitting.

9.3 What the model does not represent

The pipeline ranks by likelihood that a family is genuine, and carries no term for what would happen if it were. A critical public-facing server and a spare workstation showing identical activity receive identical treatment. This makes it a likelihood ranker. A risk score would also weigh what a compromise of that asset costs, and that gap separates the coverage metric from what an analyst actually optimises. Closing it needs asset criticality and business impact, which the dataset does not carry.

The tool also works in day-sized batches, scoring a period after it closes. A real SOC works in shifts on a live stream, and nothing here addresses streaming, mid-day scoring, or the cadence of a shift handover beyond a decision export.

Several deliberate exclusions also bound what the model can do. Machine names, addresses, rule identifiers and alert names are kept out of the feature matrix so that a model trained on seven companies

cannot recognise the eighth by name. This is necessary for the cross-environment claim and it discards information a real deployment would legitimately use, since a SOC does know which of its own machines matter. The reviewed ATT&CK mapping is likewise held back from the model because it was authored with knowledge of this dataset, and feeding it in was measured to make the queue worse regardless.

No adversarial analysis was run. Two re-ranker weights are negative, many rule groups and many families on a machine both lower a family's rank, so noise generated on a controlled host would suppress the ranking of its own real activity. The family is also bounded by one host, so a campaign spread across machines appears as unlinked queue items.

9.4 The evaluation is weaker than a significance test

Dominance is reported. With eight folds that share seven training environments each, paired tests are neither independent nor adequately powered, and none of the comparisons run clears a corrected threshold. The claims are therefore of the form "at least equal everywhere and better somewhere", which is weaker than it looks in a table.

All eight environments informed design decisions at some point in the project, so none is a genuinely untouched test set. The protocol prevents a model from seeing its test environment during training; it does not prevent me from having seen it while choosing what to build.

Absolute coverage at the smallest budget is also unstable. Repeating the evaluation at 50, 200, 300 and 500 trees and across seeds moves coverage at a budget of five by several windows, and the three-seed closure reported here spans 51 to 53. Increasing the forest size does not monotonically help, and coverage at five was higher with 50 trees than with 500 in one sweep, which is not explained. The ordering between methods is stable across all of these configurations. The specific value at the tightest budget moves by several windows.

9.5 The author's own limitations

The alternative dataset discussed in Section 2.5 would address several of the limitations above and was not used, for the reasons given there.

This work was done by a student with coursework in machine learning and security, and no operational experience in a Security Operations Centre. Every judgement about what an analyst wants is therefore inferred from published accounts. The choice of coverage as the objective, the family as the unit, the evidence layout and the ten-a-day budget are all reasoned, none validated: no analyst has used the tool or reviewed its output.

The work was also unsupervised in the academic sense. A report of this kind would normally be produced under an advisor who challenges the framing and knows which directions are dead ends; I had documentation, published papers, occasional exchanges with teachers, and large language models as a sounding board and for running tests. Four weeks of work produced this pipeline and this report, and that pace owes a lot to those tools. It should be read as a project report.

The measurements are honest and reproducible. Whether they measure the right objective for a real SOC is untested.

10 Conclusion

Meerkat normalizes heterogeneous detector output against a customer-supplied asset inventory, groups it into sessions and daily families, ranks those families under a capacity-limited queue, enriches them with auditable ATT&CK context, and is evaluated across environments it has never seen.

Ranking individual alerts fails: 20 of 79 windows at fifty reviews a day, none of the intrusion's quiet phases, and worse than sorting on native severity. Ranking alert families succeeds: 34, 43 and 55 windows at 5, 10 and 25 reviews per day. Adding a session layer and a learned re-ranker reaches 52, 58 and 58 of the 60 label-reachable windows, against 19, 33 and 46 for the detectors' own severity ordering, reading about seven times fewer alerts than the best-child ordering whose coverage comes closest. The feature set and the model class were closed by measurement, 34 of 34 removal masks rejected and gradient boosting beaten on every seed. A client refits the model on incident records behind a gate that discards any refit which does not beat the shipped bundle on their own held-out incidents, and the confidence beside each queue item is the analyst's own escalation record.

I started this project to find out what a SOC analyst's problem actually is. The answer I did not expect is that most of it is decided before any model is trained, in the definition of what counts as one thing to look at and what counts as doing well.

References

[1] M. Landauer, F. Skopik and M. Wurzenberger, *Introducing a New Alert Data Set for Multi-Step Attack Analysis*, Cyber Security Experimentation and Test Workshop (CSET), 2024.

[2] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger and A. Rauber, *Maintainable Log Datasets for Evaluation of Intrusion Detection Systems*, IEEE Transactions on Dependable and Secure Computing, 2023.

[3] MITRE, *MITRE ATT&CK Enterprise Matrix and STIX Distribution*, `https://attack.mitre.org`.

[4] S. Freitas, A. Gharib and M. Magenheim, *Adaptive Incident Prioritization for Security Operations at Scale*, arXiv:2607.16963, 2026.

[5] S. Freitas and A. Gharib, *GraphWeaver: Billion-Scale Cybersecurity Incident Correlation*, ACM International Conference on Information and Knowledge Management (CIKM), 2024.

[6] B. Gelman, S. Taoufiq, T. Vörös and K. Berlin, *That Escalated Quickly: An ML Framework for Alert Prioritization*, arXiv:2302.06648, 2023.

[7] *AlertBERT: A Noise-Robust Alert Grouping Framework for Simultaneous Cyber Attacks*, arXiv:2602.06534, 2026.

[8] Y. Liu et al., *RAPID: Real-Time Alert Investigation with Context-aware Prioritization for Efficient Threat Discovery*, Annual Computer Security Applications Conference (ACSAC), 2022.

[9] *Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods*, ACM Computing Surveys, 2024.

[10] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias and K. Li, *AI^2: Training a Big Data Machine to Defend*, IEEE International Conference on Big Data Security, 2016.

[11] Microsoft, *GUIDE: a Dataset of Real-World Security Incident Triage Decisions*, 2024. Expert incident-prioritization labels for 9,980 incidents across 499 organisation queues, with a queue-ranking evaluation protocol, added 18 July 2026.

[12] T. van Ede et al., *DeepCASE: Semi-Supervised Contextual Analysis of Security Events*, IEEE Symposium on Security and Privacy, 2022.

[13] *CAM-LDS log data set.*

[14] *AInception alert data set.*